278　ソーティングされたデータの木構造への展開

堤　正勝†　　荒木　健治††　　宮永　喜一†　　栃内　香次†
†北海道大学工学部　　††北海学園大学工学部
Email: tutumi@hudk.hokudai.ac.jp

１．はじめに

最近市中に出回っているアルゴリズム辞典などと題した書物を見ると大抵の書物には木構造についてのアルゴリズムとして、ソーティング、検索などが載っている [2][1] が、その手法の欠点として、すでにソーティングされた辞書（ここでは説明の都合上かな漢字変換に用いる単語辞書を例に話を進める。もちろんデータベースのキーワード等でもよい）をロードすると、右あるいは左どちらか一方の枝だけが伸びてしまい、登録や検索の効率が著しく低下することが書かれている。このような構造になってしまった木を再構成するアルゴリズムはすでに発表されている [4] が、静的なアルゴリズムであり、登録が完了した後で、実行される種類のものである。著者らは以前これに対して、動的にこのアルゴリズムを適用することにより、頻度の高い語をより root の方に寄せ、より適切な木を作成する手法を考案した [3]。しかしながら、この手法は検索を実際に行なっている状態でなければ機能しないものであった。本稿では、このアルゴリズムをさらに効率化するために、辞書をはじめて使用する時の、各単語に対するアクセス頻度が不明であり、すでにソーティングされている場合に、片方の枝のみが伸びてしまわないようなアルゴリズム、つまり初期のロード時にソートされたものを扱うと仮定したアルゴリズムを提案する。具体的には、ソーティングされた語数 W のデータベースに対してその深さが log2(w)+1 を越えないようにロードして行くアルゴリズムである。

２．アルゴリズム

２．１　基本概念

左側が常に完全木（全ての末端の深さが一定の木）になる様にする。右側は、再帰的に左側が完全木になる様に登録してゆき、左と同じ深さの完全木になったら、次の登録は root に行なう。root に登録される語は常に２のｎ乗（ｎは負でない整数）番目の語である。

２．２　具体例

はじめ (1) は、root = nil であるので、root に登録する。次 (2) は現在の木が深さ１の完全木なので、これを左として、新らしい語を root とする。その次の語 (3) は右の方が浅いので右へ登録する。これで、深さ２の完全木となる。次の語 (4) は完全木に対する登録なので root とし、元の木は語 (4) の左とする。語 (5) は語 (3) と同様の登録。しかし、右はまだ完全木になっていない。そこで、次の語 (6) は再帰的に語 (2) と同様に、語 (4) の右枝以下の部分木が完全木なのでこれを左とする節として、語 (6) を部分木の root とする。次の語 (7) は全く語 (3) と同様で完全木ができる。以下同様に、この操作を繰り返す。右図参照。

２．３　実用

実際の使用にあたっては、エディタなどで、一部の順序が入れ替わったり、また、ソーティングの後に追加登録されているような辞書、つまりほぼソーティングされている辞書を許すことにする。アルゴリズムは、文字列比較を行ない、チェックしながら、左側へ登録を許すようにする。

３．インプリメント

本アルゴリズムは『自然言語処理支援エディタ』における構造化辞書編集モードのプロトタイプである、構造化辞書エディタ（ｄｍ）の、既存辞書変換フィルタ [5] の中にインプリメント中である。

４．おわりに

本報告では、ソーティングされた辞書の木構造への登録手法について報告した。今後、このアルゴリズムをインプリメントし、既存アルゴリズムとの性能比較を、発表時までに行ないたいと考えている。

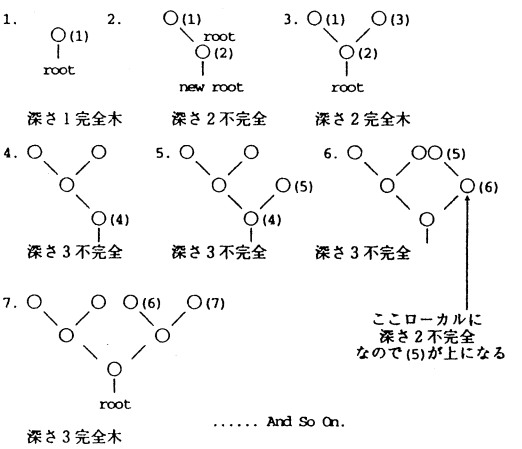


図１　単語登録の様子

参考文献

[1] Knuth:
The Art of Computer Programming 2,
サイエンス社

[2] 奥村晴彦:
Ｃ言語による最新アルゴリズム辞典,
技術評論社,1991

[3] 堤 正勝：
学習機能を有する木構造辞書について,
北海道大学工学部, 卒業論文, 昭和６３年

[4] G.M.Adel'son-Velskii, E.M.Landis:
ＡＶＬ法,1962,
(『インターフェース』,1990-4,pp173)

[5] 堤　正勝、荒木健治、宮永喜一、栃内香次:
自然言語処理支援エディタの改良－単語辞書の階層化－:
平成７年度電気関係学会北海道支部連合大会 (本誌).